

LEI - Licenciatura Engenharia Informática

Sistemas Distribuídos

Trabalho prático



**ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO**

Developed By:

João Santos – 8220256

Sónia Oliveira - 8220114

Visão geral do projeto

A aplicação desenvolvida permite a troca de mensagens e pedidos entre utilizadores dentro de 4 grupos de chat.

Foi utilizado o protocolo UDP para o envio de mensagens normais e foi usado o protocolo TCP para o tratamento de pedidos especiais por parte do servidor. Qualquer utilizador pode fazer uso destes pedidos especiais que são mostrados aquando de entrada em qualquer groupchat.

```
[2024-12-12T22:13:34.043869900] low2: has joined the all chat!  
[2024-12-12T22:13:42.435858500] low2: EMERGENCY_COMS_ACTIVATED_BY:low2:Group0  
--- End of Chat History ---
```

```
Welcome to all chat! List of commands :  
- exit: exit to leave the chat.  
- commands: to repeat this message.  
- EMERGENCY_COMS : to create a EMERGENCY_COMS notification or request  
- EMERGENCY_RESOURCES : to create a EMERGENCY_RESOURCES notification or request  
- MASS_EVACUATION : to create a MASS_EVACUATION notification or request.
```

Como podemos ver na imagem os comandos especiais que podemos utilizar são:

- **EMERGENCY_COMS** – este pedido pode ser efetuado por qualquer utilizador com perfil de LOW ou acima.

- **EMERGENCY_RESOURCES** – este pedido pode ser efetuado por qualquer utilizador com perfil de MID ou acima. Quando um utilizador do tipo LOW faz este tipo de pedido, o servidor envia para o groupchat MID e HIGH um pedido a notificar os utilizadores nesses groupchats que um utilizador com perfil LOW quer ativar os emergency resources, e deve então ativá-los se concordar com a decisão.

- **MASS_EVACUATION** – este pedido pode ser efetuado por qualquer utilizador com perfil de LOW ou acima. Quando um utilizador do tipo LOW ou MID faz este tipo de pedido, o servidor envia para o groupchat HIGH um pedido a notificar os utilizadores nesses groupchats que o utilizador que usou o comando quer ativar um mass evacuation, e deve então ativá-los se concordar com a decisão.

Como podemos ver, os utilizadores têm três tipos de permissões, LOW, MEDIUM e HIGH. Qualquer tipo de utilizador pode entrar em qualquer chat, por motivos de comunicação facilitada entre utilizadores, mas apenas os utilizadores com permissão correta podem fazer os pedidos especificados acima.

Como os pedidos foram desenvolvidos com o protocolo TCP, sempre que há um pedido, é garantido que o servidor trate dele e o envie para os utilizadores relevantes.

Troca de mensagens

A aplicação também permite a troca de mensagens normais, ou seja, não comandos entre utilizadores, nos vários groupchats. Isto é alcançado com o uso do protocolo UDP, pelo que as mensagens, podem, por vezes, não ser entregues.

```
[2024-12-12T19:53:39.214976300] joao: EMERGENCY_COMS_ACTIVATED_BY:joao:Group1
[2024-12-12T19:53:39.226302700] juno: EMERGENCY_COMS_ACTIVATED_BY:joao:Group1
[2024-12-12T19:54:05.846634500] joao: EMERGENCY_COMS_ACTIVATED_BY:joao:Group1
[2024-12-12T19:54:05.847881500] juno: EMERGENCY_COMS_ACTIVATED_BY:joao:Group1
--- End of Chat History ---

Welcome to low chat! List of commands :
- exit: exit to leave the chat.
- commands: to repeat this message.
- EMERGENCY_COMS : to create a EMERGENCY_COMS notification or request
- EMERGENCY_RESOURCES : to create a EMERGENCY_RESOURCES notification or request
- MASS_EVACUATION : to create a MASS_EVACUATION notification or request.
Server response to port notification: Notification port updated successfully
Notification listener started on port: 51686
olá !
User{name='juno', profile=HIGH}:olá !
User{name='joao', profile=HIGH}:olá juno!
como estás?
User{name='juno', profile=HIGH}:como estás?
User{name='joao', profile=HIGH}:está tudo bem e contigo?
```

As mensagens são específicas a cada groupchat, ou seja, mensagens enviadas no groupchat 1 (low) apenas serão broadcasted para utilizadores que também estejam nesse canal.

Autenticação

Aquando um utilizador inicia o seu lado do programa ele é apresentado com uma interface de utilizador na consola, onde se deve registar ou fazer login. Os logins existentes de momento encontram-se no ficheiro data/users.json.

```
{
  "low2": {
    "name": "low2",
    "profile": "LOW",
    "password": "low2",
    "usernameId": "low2"
  },
  "joao": {
    "name": "joao",
    "profile": "HIGH",
    "password": "joaojoao",
    "usernameId": "joao"
  },
  "juno": {
    "name": "juno",
    "profile": "HIGH",
    "password": "junojuno",
    "usernameId": "juno"
  }
}
```

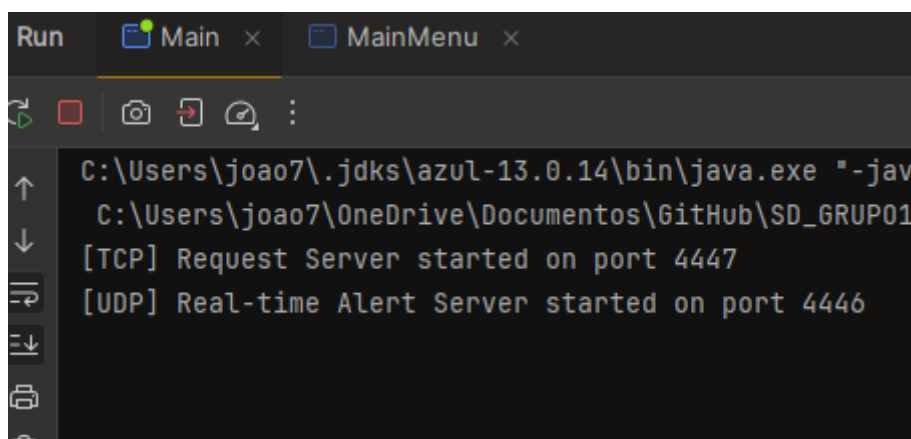
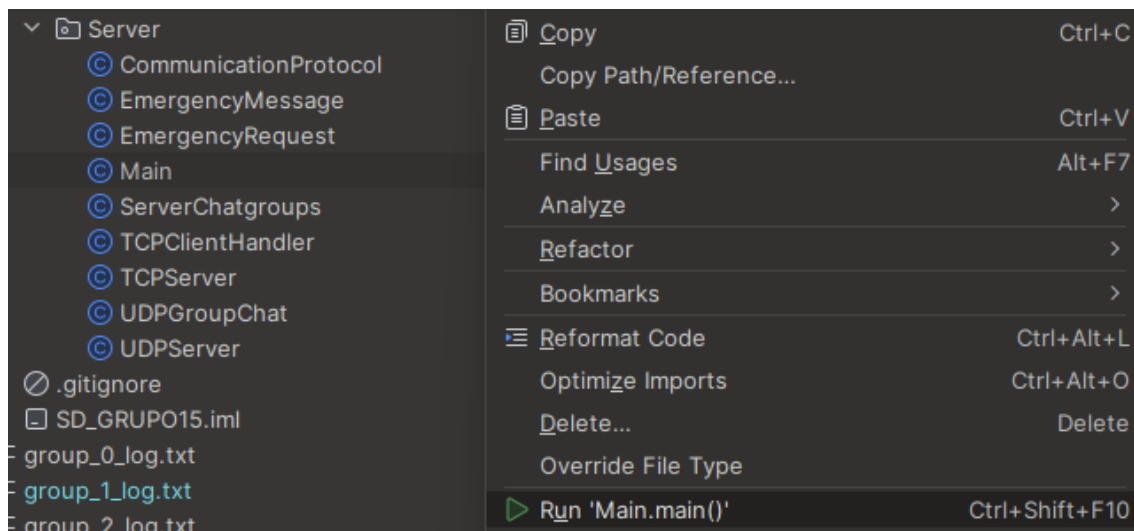
```
=== Main Menu ===
1. Register
2. Login
3. Exit
Choose an option: 2

--- Login ---
Enter user ID: juno
Enter password: junojuno
Login successful! Welcome back, juno

=== Welcome, juno ===
1. Choose Groupchat
2. Logout
Choose an option: 1
Please enter The Groupchat you wish to enter, keep in mind
1- all chat
2 - low chat
3 - medium chat
4 - high chat all other numbers will be invalid.
2
```

Passos de utilização

Para correr o programa corretamente, o utilizador **deve primeiro lançar o servidor e verificar que o mesmo está a correr e ativo. Este pode ser encontrado no método main na pasta Server.**



Quando o servidor se encontra a correr, podemos lançar o número desejado de MainMenus que se encontram na pasta menus, para simular múltiplos utilizadores online ao mesmo tempo. Sempre que iniciamos o método main da class MainMenu, somos apresentados com a interface de utilizador que devemos navegar, fazendo login, escolhendo o groupchat, e depois enviando mensagens conforme desejado.

Explicação do servidor

O servidor deste programa faz uso de duas threads que simulam dois servidores separados, um para TCP e outro UDP que partilham um contexto que inclui os groupchats e utilizadores ligados nos mesmos. Isto pode ser visto na classe Main da pasta Server.

Servidor TCP

O **TCP Server** possui o port 4447, contexto e um executor service para lançar várias threads. Quando lançamos o método start, começamos o servidor TCP que vai criar uma socket no porto 4447 e esperar por uma conexão do cliente. Quando um cliente se conecta, uma thread vai ser lançada numa thread separada utilizando a classe executor service. Existem vários logs dentro da classe TCP servidor para efeitos de mostrar o funcionamento da lógica do programa. Quando um cliente se liga, o método handleClient no TCPServer é responsável por ler e responder a pedidos do cliente. O método processa o pedido do cliente, recebendo-o, verificando o formato e executando a resposta apropriada com base nos detalhes recebidas na mensagem, que representam comandos.

O método handleClient recebe primeiramente um pedido sem o utilizador saber quando este se conecta a um groupchat, onde recebe a porta do cliente, e se liga à mesma. Cada cliente possui uma listenerSocket para receber notificações por parte do servidor. Isto acontece pois quando se faz um pedido, os utilizadores são todos informados. Ou seja, quando há por exemplo, um MASS_EVACUATION lançado por um utilizador com perfil HIGH, o servidor vai fazer uso da ligação à listenerSocket de cada utilizador e enviar uma notificação a avisar que um MASS_EVACUATION foi ativado, pelo utilizador responsável, e no grupo em que foi ativado. Os comandos podem ser lançados desde qualquer groupchat, desde que o utilizador tenha perfil alto o suficiente para o lançar.

Servidor UDP

O **TCP Server** possui o port 4446 e contexto, lança uma `datagramSocket` para ficar à espera de mensagens por parte dos clientes. Quando recebe uma, verifica o seu formato, que espera receber no formato "USERNAME:GROUPID:MESSAGE:", e se for válida, vai fazer multicast da mesma mensagem para todos os utilizadores no groupchat, excepto o utilizador que lançou a mensagem.

Protocolo de comunicação

ambos os servidores esperam receber mensagens no formato
"Username:GROUPID:MENSAGEM"

```
default:
    String chatMessage = loggedInUser + ":3:" + message;
    byte[] chatData = chatMessage.getBytes();
    DatagramPacket chatPacket = new DatagramPacket(chatData, chatData.length, serverAddress, serverPortUDP);
    socket.send(chatPacket);
    break;
```

Com a exceção do `setNotificationPort` no `servidorTCP`

```
if (parts.length >= 4 && parts[2].equals("SET_NOTIFICATION_PORT")) {
    String username = parts[0];
    int notificationPort = Integer.parseInt(parts[3]);

    // Find and update the user's notification port
    System.out.println("this is what I'm sending to findOrCreateUserInfo method");
    UserInfo user = findOrCreateUserInfo(username, parts[1], clientSocket);
    user.setNotificationPort(notificationPort);

    output.println("Notification port updated successfully");
    return;
}
```